

# SESSION 1 — TRAVAUX PRATIQUES

## Construire un assistant de recherche documentaire

---

### Objectif

Créer un outil en ligne de commande qui permet de charger un document texte et de poser des questions dessus. L'outil doit retrouver les passages pertinents et générer une réponse basée uniquement sur le contenu du document.

Ce TP pose les bases de ce qu'on appelle **RAG (Retrieval-Augmented Generation)** une technique fondamentale qu'on approfondira en session 3.

---

### Pourquoi c'est utile

Imaginons que tu veuilles interroger :

- Un contrat de 50 pages pour savoir si une clause spécifique existe
- Une documentation technique pour trouver comment configurer une fonctionnalité
- Un rapport annuel pour extraire des chiffres précis

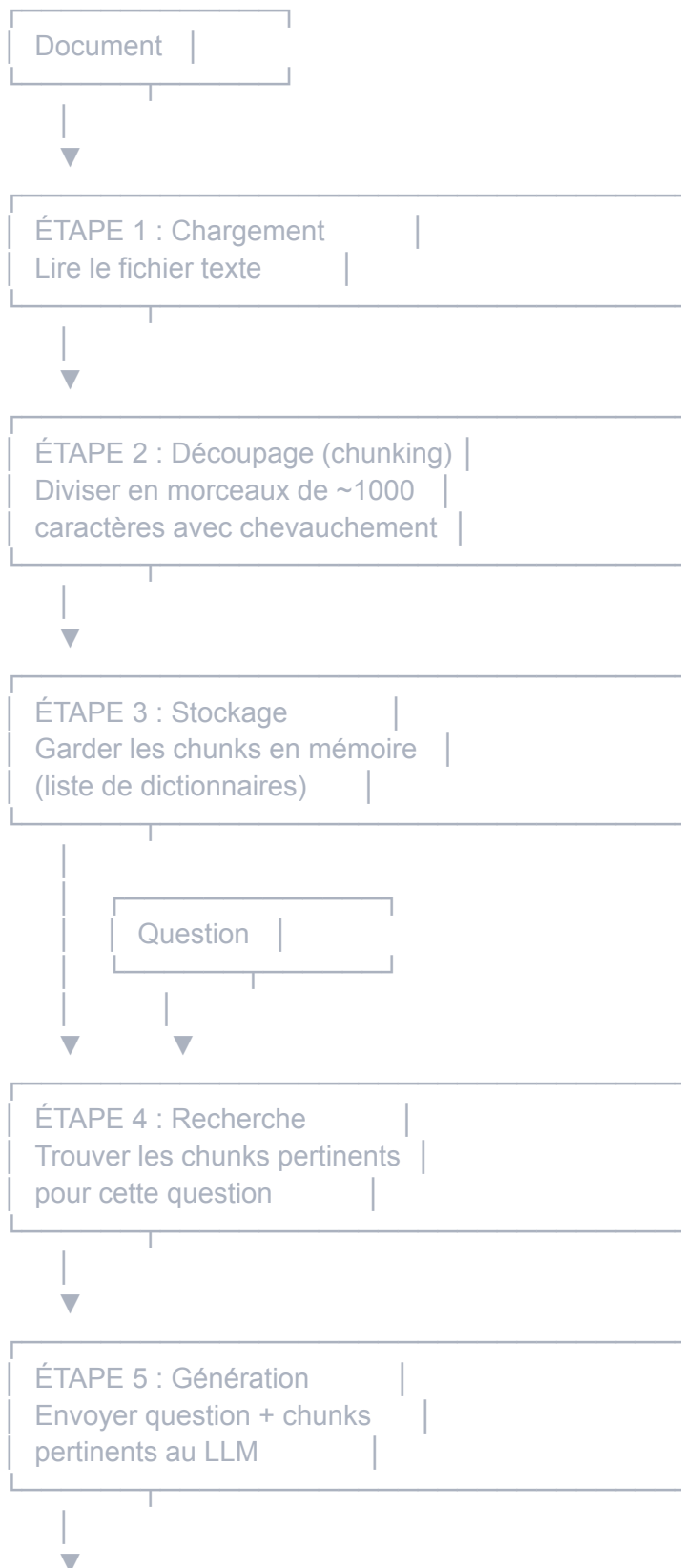
Tu pourrais copier-coller tout le document dans Claude et poser ta question. Mais :

1. Ça coûte cher en tokens (tu paies pour tout le document à chaque question)
2. Le document peut dépasser la limite de contexte
3. Même si ça tient, le modèle risque de "perdre" l'information au milieu (Lost in the Middle)

La solution : découper le document en morceaux, trouver les morceaux pertinents pour chaque question, et n'envoyer que ceux-là au modèle.

---

## Architecture cible



## Étape 1 : Chargement du document

**Objectif** : Écrire une fonction qui prend un chemin de fichier et retourne son contenu sous forme de chaîne de caractères.

**Ce qu'il faut gérer** :

- Fichiers `.txt` et `.md` (lecture simple en UTF-8)
- Gestion des erreurs si le fichier n'existe pas

**Bonus** : Support des fichiers PDF avec la librairie `pymupdf` (à installer avec `pip install pymupdf`).

**Critère de réussite** : La fonction retourne le texte complet du document.

---

## Étape 2 : Découpage en chunks

**Objectif** : Écrire une fonction qui prend un texte long et le découpe en morceaux de taille raisonnable.

**Paramètres suggérés** :

- `chunk_size` : taille cible de chaque morceau (~1000 caractères)
- `overlap` : chevauchement entre morceaux consécutifs (~200 caractères)

### Pourquoi le chevauchement ?

Imagine cette phrase coupée en plein milieu :

Chunk 1: "...l'employé a droit à 3 jours de télétravail"

Chunk 2: "par semaine, sauf accord contraire du manager..."

Sans overlap, si quelqu'un demande "combien de jours de télétravail ?", l'info est coupée en deux et peut être mal interprétée. Avec un overlap de 200 caractères, le chunk 2 contiendrait aussi "3 jours de télétravail par semaine" en entier.

**Amélioration importante** : Ne pas couper au milieu d'une phrase. Chercher le dernier point, point d'exclamation, point d'interrogation ou retour à la ligne dans les ~200 derniers caractères du chunk, et couper là.

**Format de sortie suggéré** : Une liste de dictionnaires, chacun contenant :

- **content** : le texte du chunk
- **index** : son numéro (pour pouvoir le citer dans la réponse)

**Critère de réussite** : Un document de 10 000 caractères est découpé en environ 10-12 chunks qui se chevauchent légèrement.

---

### Étape 3 : Recherche des chunks pertinents

**Objectif** : Étant donné une question et une liste de chunks, trouver les 3 chunks les plus pertinents pour répondre à cette question.

**Approche simple (qu'on utilisera ici)** : Demander au LLM de scorer chaque chunk.

Pour chaque chunk, envoyer un prompt du type :

Sur une échelle de 0 à 10, quelle est la pertinence de ce passage pour répondre à la question suivante ?

Question : [la question de l'utilisateur]

Passage : [le contenu du chunk, tronqué si trop long]

Réponds UNIQUEMENT par un nombre entre 0 et 10.

Récupérer le score, trier les chunks par score décroissant, garder les 3 meilleurs.

**Pourquoi c'est lent** : On fait un appel API par chunk. Pour 50 chunks, ça fait 50 appels. En session 3, on verra les **embeddings vectoriels** qui permettent de faire cette recherche en millisecondes sans appel API.

**Critère de réussite** : Pour une question sur le télétravail dans un document RH, les chunks retournés parlent effectivement de télétravail.

---

### Étape 4 : Génération de la réponse

**Objectif** : Envoyer au LLM la question de l'utilisateur accompagnée des chunks pertinents, et obtenir une réponse.

### Construction du prompt :

Le **system prompt** doit cadrer le comportement :

- Répondre UNIQUEMENT à partir des extraits fournis
- Si l'information n'est pas dans les extraits, le dire clairement
- Citer les numéros d'extraits quand on fait référence à une information
- Être précis et concis

Le **user prompt** doit contenir :

- Les chunks pertinents, clairement séparés et numérotés
- La question de l'utilisateur

### Exemple de structure :

Voici des extraits d'un document :

[Extrait 1]

...contenu du chunk 1...

[Extrait 2]

...contenu du chunk 2...

[Extrait 3]

...contenu du chunk 3...

---

Question : [question de l'utilisateur]

**Critère de réussite** : La réponse est basée sur le contenu des extraits, cite les numéros d'extraits, et indique clairement si l'information demandée n'est pas disponible.

---

## Étape 5 : Interface utilisateur

**Objectif** : Créer une boucle interactive où l'utilisateur peut poser plusieurs questions sur le même document.

### Fonctionnalités :

- Charger le document une seule fois au démarrage
- Afficher des informations sur le document (nombre de caractères, nombre de chunks)
- Boucle de questions/réponses
- Commande pour quitter (ex: "quit" ou "exit")
- Optionnel : afficher quels chunks ont été sélectionnés (mode verbose)

**Critère de réussite** : On peut charger un document et poser 5 questions d'affilée sans recharger.

---

## Document de test

Utilisez ce document pour tester votre implémentation :

Politique de télétravail de l'entreprise TechCorp  
Version 2.3 - Janvier 2026

### ARTICLE 1 : ÉLIGIBILITÉ

Le télétravail est accessible à tous les employés ayant terminé leur période d'essai, soit 3 mois pour les CDI et 1 mois pour les CDD. Les stagiaires ne sont pas éligibles au télétravail pendant les 2 premiers mois de leur stage.

Les postes nécessitant une présence physique obligatoire (accueil, maintenance, sécurité) ne sont pas éligibles au télétravail régulier, mais peuvent bénéficier de télétravail occasionnel avec accord du manager.

### ARTICLE 2 : MODALITÉS

Le télétravail régulier est limité à 3 jours par semaine maximum. Les jours de télétravail doivent être fixés en accord avec le manager et l'équipe, en tenant compte des réunions d'équipe obligatoires en présentiel le mardi.

Le télétravail occasionnel (maximum 10 jours par an) peut être demandé avec un préavis de 48 heures, sauf cas de force majeure (intempéries, grèves de transport, maladie d'un enfant).

### ARTICLE 3 : ÉQUIPEMENT

L'entreprise fournit un ordinateur portable et un casque audio à tout télétravailleur régulier. Une participation de 50€ par mois est versée pour les frais de connexion internet et d'électricité, sur présentation des justificatifs.

Le mobilier (bureau, chaise) reste à la charge de l'employé. Une aide exceptionnelle

de 200€ peut être accordée sur demande motivée auprès des RH.

#### ARTICLE 4 : HORAIRES ET DISPONIBILITÉ

Le télétravailleur doit être joignable pendant les plages horaires fixes : 9h30-12h00 et 14h00-17h00. En dehors de ces plages, la flexibilité est accordée dans le respect de la durée légale du travail.

Le droit à la déconnexion s'applique : aucune réponse n'est attendue après 19h00 et le week-end, sauf situation d'urgence définie par le manager.

#### ARTICLE 5 : SANTÉ ET SÉCURITÉ

Le télétravailleur doit attester sur l'honneur que son espace de travail à domicile respecte les normes de sécurité électrique et ergonomiques minimales.

En cas d'accident du travail à domicile pendant les heures de travail déclarées, les mêmes protections s'appliquent qu'en présentiel. L'accident doit être déclaré dans les 24 heures au service RH.

#### ARTICLE 6 : CONFIDENTIALITÉ

Le télétravailleur s'engage à travailler dans un environnement garantissant la confidentialité des données de l'entreprise. L'utilisation d'un VPN est obligatoire pour tout accès aux systèmes internes.

Le travail dans des espaces publics (cafés, espaces de coworking non sécurisés) est déconseillé pour les tâches impliquant des données sensibles.

#### ARTICLE 7 : RÉVERSIBILITÉ

Le télétravail est réversible à tout moment, à l'initiative de l'employé ou de l'entreprise, avec un préavis de 2 semaines. En cas de non-respect des obligations du présent accord, la réversibilité peut être immédiate.

#### Questions de test :

1. "Combien de jours de télétravail par semaine sont autorisés ?"
2. "Un stagiaire peut-il faire du télétravail ?"
3. "Quelle est l'aide pour le mobilier ?"
4. "Quelles sont les plages horaires obligatoires ?"
5. "Que se passe-t-il en cas d'accident à domicile ?"
6. "Est-ce que je peux travailler depuis un café ?" (réponse nuancée attendue)
7. "Quel est le salaire minimum ?" (information absente — le modèle doit le dire)

---

## Extensions (pour ceux qui finissent en avance)

### Extension 1 — Historique de conversation

Modifier le système pour garder en mémoire les questions et réponses précédentes. Les inclure dans le prompt pour que le modèle puisse faire référence à ce qui a été dit avant.

Exemple :

- Q1 : "Combien de jours de télétravail ?" → R1 : "3 jours maximum"
- Q2 : "Et pour les stagiaires ?" → Le modèle doit comprendre qu'on parle toujours de télétravail

### Extension 2 — Comparaison de modèles

Modifier le code pour poser la même question à Claude ET à GPT, et afficher les deux réponses côte à côte. Observer les différences de style, de précision, de concision.

### Extension 3 — Résumé automatique

Ajouter une commande "résumé" qui génère un résumé du document entier. Approche : résumer chaque chunk individuellement, puis faire un résumé des résumés.

### Extension 4 — Votre propre document

Tester avec un vrai document de votre choix : un cours, une documentation technique, un contrat, un article long. Observer comment le système se comporte avec du contenu réel.

## Ce que ce TP vous apprend

1. **Le chunking** : Un document long doit être découpé intelligemment — ni trop petit (perte de contexte), ni trop grand (coût et "lost in the middle")
2. **L'overlap** : Le chevauchement évite de couper les informations en plein milieu
3. **La recherche sémantique** : On ne peut pas tout envoyer au modèle, il faut sélectionner. Ici on utilise le LLM lui-même — en session 3 on verra les embeddings, beaucoup plus rapides
4. **Le prompt engineering** : Cadrer précisément ce qu'on attend du modèle ("uniquement ces extraits", "cite tes sources", "dis si tu ne sais pas")
5. **Les limites** : Le système ne peut répondre que sur ce qui est dans le document. C'est une feature, pas un bug — ça évite les hallucinations

**Bon courage !**